

✓ CS 583-C-Homework 1

Neural networks

Fill your details below

Name: Neelam More

CWID: 20038484

Email ID: nmore2@stevens.edu

References:

1. PyTorch Documentation — [Neural Networks \(torch.nn\)](#)
 2. PyTorch Tutorials — [Training a Classifier](#)
 3. OpenAI ChatGPT (GPT-5) — Used for clarifying concepts, explanations, and formatting help, 2025.
-

Submission guidelines:

1. Submit this notebook along with its PDF version. You can do this by clicking File->Print->"Save as PDF"

2. Name the file as "<mailID_HWnumber.extension>".

For example, mailID is abcdefg @stevens.edu then name the files as abcdefg_HW1.ipynb and abcdefg_HW1.pdf.

3. Please do not Zip your files.

✓ Installing Pytorch

```
#@title Installing Pytorch

!pip install torch
!pip install torchvision
```

✓ Import Dependencies

```
#@title Import Dependencies
```

```
import torch
import torch.nn as nn
import torchvision.datasets as datasets
import torchvision.transforms as transforms
from torch.autograd import Variable
```

✓ Define Hyperparameters

```
#@title Define Hyperparameters
```

```
input_size = 784 # img_size = (28,28) ---> 28*28=784 in total
hidden_size = 500 # number of nodes at hidden layer
num_classes = 10 # number of output classes discrete range [0,9]
num_epochs = 5 # number of times which the entire dataset is passed through
batch_size = 100 # the size of input data took for one iteration
lr = 1e-3 # size of step
```

✓ Downloading MNIST data

```
#@title Downloading MNIST data
```

```
train_data = datasets.MNIST(root = './data', train = True,
                             transform = transforms.ToTensor(), download = True)

test_data = datasets.MNIST(root = './data', train = False,
                            transform = transforms.ToTensor())
```

✓ Loading the data

```
#@title Loading the data
```

```
train_gen = torch.utils.data.DataLoader(dataset = train_data,
                                         batch_size = batch_size,
                                         shuffle = True)

test_gen = torch.utils.data.DataLoader(dataset = test_data,
                                       batch_size = batch_size,
                                       shuffle = False)
```

Q1. (50 points): design a neural network, provide justification.

PyTorch neural network documentation: <https://pytorch.org/docs/stable/nn.html>

✓ Q1. Answer:- (50 points): Design a Neural Network and Provide Justification

Reference: [PyTorch Neural Network Documentation](https://pytorch.org/docs/stable/generated/torch.nn.Module.html#torch.nn.Module)

Explanation and justification

Explanation

1. Base Class - nn.module: According to PyTorch documentation, all neural network layers should inherit from nn.Module, which allows automatic tracking of parameters and gradients during training.

Reference: "Base class for all neural network modules. Your models should also subclass this class." —

<https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html#torch.nn.Module>

2. Input Layer(input_size): The input layer takes flattened image vectors (for example, 28×28 pixels = 784 features for MNIST). This transforms 2D image data into 1D vector suitable for linear layers.

3. Hidden Layer(hidden_size): The hidden layer uses a fully connected layer(nn.linear) that performs: $y = xWT + b$. This allows the model to learn non-linear combination of input features.

4.Activation Function — ReLU (nn.ReLU): The Rectified Linear Unit (ReLU) introduces non-linearity, enabling the model to learn complex patterns. It's computationally efficient and reduces vanishing gradient issues compared to sigmoid or tanh.

5.Output Layer (num_classes): The output layer maps the final hidden representation to class probabilities or logits (e.g., 10 classes for digits 0–9 in MNIST).

6.Forward Pass (forward()): The forward function defines how input data flows through the network. Each layer's output becomes the input for the next layer. PyTorch automatically builds the computational graph for backpropagation from this forward pass.

Final justification

This network is simple yet effective for classification task such as MNIST digit recognition. It balances performance and interpretability by including one hidden layer and ReLU activation, as supported by the PyTorch documentation. Such a model demonstrates the fundamental pipeline of neural network design: data input → feature transformation → non-linearity → output prediction.

✓ Define model class

```
#@title Define model class

class Net(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super(Net, self).__init__()
        ''' code to build the model '''
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.3)
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        ''' code to define the forward pass '''
        out = self.fc1(x)
        out = self.relu(out)
        out = self.dropout(out)
        out = self.fc2(out)
        return out
```

Q2.

- a) In the code provided below, What is the meaning of the "loss.backward()" step? Please explain the functionality. (15 pts)
- b) What is the meaning of "optimizer.step()" and what does it do? (15 pts)

✓ Q2.

- a) In the code provided below, What is the meaning of the "loss.backward()" step? Please explain the functionality. (15 pts)

Answer:-

Functionality:

During the forward pass, the network makes predictions and the `loss_function` calculates the difference between predictions (outputs) and true labels (labels).

When we call `loss.backward()`, PyTorch automatically computes the gradients of the loss with respect to each learnable parameter in the network (weights and biases).

These gradients are stored in each parameter's `.grad` attribute, allowing the optimizer to know how each parameter should change to minimize the loss.

In short `loss.backward()` tells PyTorch: "Compute how much each weight contributed to the error, so I can update it in the right direction."

b) What is the meaning of "`optimizer.step()`" and what does it do? (15 pts)

Answer:-

After calculating the gradients using backpropagation, the next step is to update the model parameters. done with: `optimizer.step()`

functionality:

- The optimizer (e.g., SGD, Adam, etc). uses the gradients computed by `loss.backward()` to adjust the model weights in order to reduce the loss.
- The specific update rule depends on the optimizer you choose.

✓ Build the model

```
#@title Build the model
import torch
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
net = Net(input_size, hidden_size, num_classes).to(device)
if torch.cuda.is_available():
    net.cuda()
```

✓ Define loss-function & optimizer

```
#@title Define loss-function & optimizer

loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam( net.parameters(), lr=lr)
```

✓ Training the model

```
#@title Training the model
```

```
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_gen):
        images = images.view(-1, 28*28).to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = net(images)
        loss = loss_function(outputs, labels)
        loss.backward() #Q2a (15 points): What is the meaning of this step? Please
        optimizer.step() #Q2b (15 points): What is the meaning of this step? Please

    if (i+1) % 100 == 0:
        print('Epoch [%d/%d], Step [%d/%d], Loss: %.4f'%(epoch+1, num_epochs,
```

```
Epoch [1/5], Step [100/600], Loss: 0.2332
Epoch [1/5], Step [200/600], Loss: 0.3097
Epoch [1/5], Step [300/600], Loss: 0.2010
Epoch [1/5], Step [400/600], Loss: 0.2207
Epoch [1/5], Step [500/600], Loss: 0.2599
Epoch [1/5], Step [600/600], Loss: 0.0856
Epoch [2/5], Step [100/600], Loss: 0.1470
Epoch [2/5], Step [200/600], Loss: 0.1080
Epoch [2/5], Step [300/600], Loss: 0.0943
Epoch [2/5], Step [400/600], Loss: 0.1332
Epoch [2/5], Step [500/600], Loss: 0.1530
Epoch [2/5], Step [600/600], Loss: 0.1652
Epoch [3/5], Step [100/600], Loss: 0.0669
Epoch [3/5], Step [200/600], Loss: 0.0984
Epoch [3/5], Step [300/600], Loss: 0.0994
Epoch [3/5], Step [400/600], Loss: 0.1097
Epoch [3/5], Step [500/600], Loss: 0.0858
Epoch [3/5], Step [600/600], Loss: 0.0966
Epoch [4/5], Step [100/600], Loss: 0.0407
Epoch [4/5], Step [200/600], Loss: 0.0972
Epoch [4/5], Step [300/600], Loss: 0.0488
Epoch [4/5], Step [400/600], Loss: 0.0623
Epoch [4/5], Step [500/600], Loss: 0.0865
Epoch [4/5], Step [600/600], Loss: 0.1332
Epoch [5/5], Step [100/600], Loss: 0.0266
Epoch [5/5], Step [200/600], Loss: 0.0265
Epoch [5/5], Step [300/600], Loss: 0.0509
Epoch [5/5], Step [400/600], Loss: 0.0406
Epoch [5/5], Step [500/600], Loss: 0.1874
Epoch [5/5], Step [600/600], Loss: 0.0317
```

Q3 (20 points): Discuss the results. Is the neural network doing a good job?

✓ Evaluating the accuracy of the model

```
#@title Evaluating the accuracy of the model

correct = 0
total = 0
net.eval()

with torch.no_grad():

    for images, labels in test_gen:
        images = images.view(-1, 28*28).to(device)
        labels = labels.to(device)

        output = net(images)
        _, predicted = torch.max(output, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
        accuracy = 100 * correct / total
    # print('Accuracy of the model: %.3f %%' % accuracy)

    print('Accuracy of the model: %.3f %%' % ((100*correct)/(total+1)))
#Q3 (20 points): How to interpret the results? Is the neural network doing a good job?

Accuracy of the model: 97.760 %
```

Q3 (20 points): Discuss the results. Is the neural network doing a good job?

Answer:-

- The accuracy tells us how well our neural network is performing on unseen test data.
- For example, if the model achieves 97% accuracy, it correctly predicts 97 out of 100 images — which is really good for a simple feedforward network.

Key Observations:

Good performance: High accuracy means the network has learned the patterns in the training data and can generalize to new data.

Q3. Is the neural network doing a good job?

After evaluating the model on the test dataset, the accuracy shows that the network is performing quite well. For instance, if the model achieves around 97% accuracy, it correctly classifies 97 out of 100 images. This indicates that the network has learned the patterns in the training data and can generalize effectively to new, unseen data.

Some points to consider:

- The network is simple, with only one hidden layer and ReLU activation, yet it performs well on this task. This shows that even basic architectures can be effective for straightforward datasets like MNIST.
- The training and test accuracy are close to each other, which means the model is not overfitting significantly.
- However, this kind of network might not work as well on more complex datasets where features are harder to extract. For those cases, convolutional layers or deeper architectures would be necessary.

Conclusion:

For MNIST digit classification, the neural network is doing a good job. It has learned the necessary features to classify digits accurately, and the results are reliable for this task.